BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.* Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

What a Table-Driven Agent is: It uses a giant lookup table (like a dictionary). Every time it sees something, it checks the table to see what move to make based on everything it has seen so far.
Why it is impossible for chess:
There are way too many possible moves and board positions (over 10^120 combinations!).
This is called combinatorial explosion (the numbers explode and become too huge).
No computer in the world has enough hard drive or RAM space to store a table that large.

What happens as the agent's lifetime increases ,
The longer the agent runs (more turns), the history gets longer, and the table size

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Condition: The boolean flag extracted from the current sensory input (e.g., percept['food_here'] or percept['wall_ahead']).
Action: The returned motor command (e.g., 'Suck', 'Left', 'Right').
In line with the lecture, these rules map directly from the immediate percept to an action (
f:P - >A
f:P- >A) without an __init__ method or any stored history of previous steps.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The agent got stuck because the environment is partially observable and the agent has no memory.

Because the agent can only see one tile ahead instead of the full grid or its coordinates, hitting the bottom of the U-trap looks identical to hitting the side wall. In both cases, the sensor simply says wall ahead is true.

Since the agent does not save any history, it has no idea where it just came from or that it just turned left a second ago. Every time it hits a wall, the rule forces it to turn left, step forward, hit another wall, and turn left again. Because the same input always produces the same reaction, it gets locked into an endless loop inside the U-trap and cannot find its way out.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In the model-based agent, I added an internal state using variables like visited cells, current direction, and last action.

The transition model works by taking the last action the agent performed and updating its internal position and facing direction before picking the next move. Every time it takes a step, it adds the new position into its visited set.

When the agent runs into a wall, instead of blindly turning left, the sensor and memory check if the left tile has already been visited. If left is already visited, the rule tells the agent to turn right instead. This allows the agent to recognize that going left would lead to a loop it has already seen, letting it take the unvisited exit and escape the trap.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

FACULTY OF COMPUTING